MODEL DEVELOPMENT DOCUMENT

Fiverr Classification Project: Imbalanced Binary Classification Using Machine Learning and SMOTE

Project Title: Comparative Analysis and Development of a Machine Learning Classification Model

Dataset: fiverr_data.csv

Target Variable: label

Selected Model: Tuned Random Forest Classifier with SMOTE

Classification Threshold: 0.60

Project Type: Supervised Machine Learning — Binary Classification

---

PAGE 1: EXECUTIVE SUMMARY

1.1 Introduction

This project focuses on developing a supervised machine learning classification system using the dataset fiverr_data.csv. The primary objective is to identify patterns in the available features and predict the target variable, label, which represents one of two classes: 0 or 1.

The dataset contains approximately 458,798 observations and 53 columns, including the target variable, a unique user identifier, and 51 numerical predictor features. The target classes are highly imbalanced, with approximately 97.31% of observations belonging to class 0 and 2.69% belonging to class 1.

This imbalance presents a significant modeling challenge. A classifier that predicts every observation as class 0 could achieve approximately 97.31% accuracy while failing to identify any positive cases. Consequently, accuracy alone is insufficient for evaluating model quality.

To address this challenge, the project uses Synthetic Minority Over-sampling Technique (SMOTE) to increase minority-class representation during model training. Multiple classification algorithms are evaluated, and their performance is assessed using precision, recall, F1-score, ROC-AUC, PR-AUC, confusion matrices, and training-versus-test comparisons.

The models investigated include Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, Linear Support Vector Machine, Gaussian Naive Bayes, and Extra Trees.

The Random Forest classifier emerged as the strongest initial candidate. Hyperparameter tuning was subsequently conducted to improve its predictive performance. The classification threshold was also optimized using a separate validation set.

1.2 Final Model Results

The tuned Random Forest model, trained using SMOTE and evaluated at a classification threshold of 0.60, achieved the following test-set results:

Evaluation Metric	Result
Test Accuracy	98.52%
Precision — Class 1	78.58%
Recall — Class 1	61.49%
F1-score — Class 1	68.99%
ROC-AUC	0.9564
PR-AUC	0.7203

These results indicate that the model can distinguish between the two classes effectively while achieving a useful balance between positive prediction accuracy and positive-case detection.

However, the training F1-score of 75.74% is higher than the test F1-score of 68.99%, indicating a generalization gap that should be considered when interpreting the final model.

1.3 Key Project Outcomes

The project demonstrates the following outcomes:

- Development of an end-to-end classification workflow.
- Identification and treatment of class imbalance using SMOTE.
- Comparison of multiple supervised classification algorithms.
- Application of stratified sampling and cross-validation.
- Hyperparameter optimization of the leading Random Forest model.
- Selection of a classification threshold based on validation performance.
- Evaluation of model generalization using independent test observations.
- Documentation of performance metrics, limitations, and future improvements.

The selected model is a promising candidate for further validation and application-specific evaluation.

PAGE 2: BUSINESS OBJECTIVE AND PROBLEM STATEMENT

2.1 Problem Statement

The objective of this project is to develop a machine learning classifier capable of predicting whether an observation belongs to class 0 or class 1 using the information available in the dataset.

The prediction problem can be represented mathematically as:

$$
\hat{y}=f(X)
$$

where:

- X represents the input features.
- y represents the actual target label.
- f represents the trained classification algorithm.
- $\hat{y}$ represents the predicted class.

For models that generate probabilities, the classification decision is determined using a threshold t:

$$
\hat{y} =
\begin{cases}
1, & P(y=1 \mid X) \geq t \\
0, & P(y=1 \mid X) < t
\end{cases}
$$

In this project, the selected threshold is 0.60.

The business meaning of class 0 and class 1 must be confirmed from the dataset documentation before the model is deployed. The technical analysis treats class 1 as the minority or positive class.

2.2 Project Objectives

The major objectives are:

1. Understand the dataset structure and identify data quality issues.
2. Perform exploratory data analysis to understand feature distributions and target imbalance.
3. Identify missing values, potential outliers, and highly skewed features.

4. Prepare the dataset for supervised learning.
5. Apply an appropriate technique to address class imbalance.
6. Train multiple classification algorithms.
7. Compare models using appropriate classification metrics.
8. Optimize the hyperparameters of the strongest candidate.
9. Select an appropriate classification threshold.
10. Evaluate generalization and identify possible overfitting.
11. Document the final model's strengths, limitations, and future improvements.

2.3 Why Class Imbalance Matters

The target variable contains a large majority of class 0 observations and a small minority of class 1 observations.

A model trained without appropriate consideration of this imbalance may favor the majority class because predicting class 0 frequently can minimize the overall classification error.

For example, consider a hypothetical classifier that predicts every observation as class 0. Its accuracy would be approximately 97.31%, but its recall for class 1 would be zero.

Such a classifier would be unsuitable when identifying minority-class observations is important.

Therefore, this project prioritizes metrics that directly measure positive-class detection and the trade-off between false positives and false negatives.

2.4 Project Scope

The scope includes data preparation, exploratory analysis, supervised model development, imbalance handling, model evaluation, and hyperparameter tuning.

The scope does not include production deployment, real-time prediction services, business-process integration, or confirmation of the target's real-world interpretation. These activities require additional requirements and validation.

PAGE 3: DATASET DESCRIPTION AND EXPLORATORY DATA ANALYSIS

3.1 Dataset Overview

The dataset is stored in the file fiverr_data.csv.

The initial inspection identified approximately 458,798 observations and 53 columns. The columns include:

- One target variable: label.
- One identifier: user_id.
- Fifty-one numerical predictor features.

The predictor columns use anonymized names, such as X1, X2, and similar feature labels. As their domain definitions are not available, feature-specific business interpretations cannot be reliably established from the column names alone.

The target variable contains two classes, 0 and 1.

3.2 Initial Data Inspection

The following Python functions were used to understand the dataset:

- head() to inspect sample records.
- info() to examine data types and non-null counts.
- describe() to generate descriptive statistics.
- isnull().sum() to identify missing values.
- duplicated().sum() to inspect duplicate rows.
- nunique() to examine the number of unique values in each column.

These checks provide an initial understanding of the dataset's structure, data quality, and potential preprocessing requirements.

3.3 Target Class Distribution

The class distribution is highly imbalanced.

Target Class	Approximate Percentage
Class 0	97.31%
Class 1	2.69%

This distribution motivated the use of SMOTE during model development.

The minority class is especially important because a model that performs well on class 0 but poorly on class 1 may achieve misleadingly high accuracy.

3.4 Feature Distribution Analysis

Histograms were generated to inspect the distributions of numerical features.

Histograms help identify:

- Concentrations of observations within specific ranges.

- Potentially skewed distributions.
- Unusual values.
- Features with limited variation.
- Differences in scale across features.

Boxplots were also generated to identify potential outliers.

Because the dataset is large, a sample of observations was used for visualization to reduce plotting time. Model training continued to use the relevant training dataset rather than the visualization sample.

3.5 Correlation Analysis

A correlation heatmap was generated to examine relationships between numerical features.

Correlation analysis helps identify:

- Strong positive relationships.
- Strong negative relationships.
- Potential multicollinearity.
- Groups of features that may contain overlapping information.

The pairplot visualization was restricted to a small number of features selected using their absolute correlation with the target.

Correlation should not be interpreted as proof of causation. Additionally, low correlation does not necessarily mean that a feature lacks predictive value, particularly when its relationship with the target is nonlinear.

3.6 EDA Findings

The principal findings were:

1. The dataset is large enough to support multiple supervised classification experiments.
2. The target distribution is highly imbalanced.
3. The user_id column is an identifier rather than a meaningful predictor and was excluded from model training.
4. Missing values required preprocessing.
5. Potential outliers were identified for further consideration.

6. Feature distributions and scales justified evaluating appropriate preprocessing strategies.
7. The anonymized predictor names limited domain-specific interpretation.

These findings informed the subsequent data preparation and modeling stages.

PAGE 4: DATA CLEANING AND FEATURE PREPROCESSING

4.1 Missing Value Treatment

Initial inspection identified six missing entries in the dataset.

Missing values can cause errors in many machine learning algorithms or reduce the number of usable observations if rows are removed indiscriminately.

The project therefore used median imputation for numerical predictors.

The median is less sensitive to extreme values than the mean and is a reasonable baseline for numerical data with possible skewness.

Importantly, the imputer was placed inside the modeling pipeline. It was fitted using the relevant training observations and subsequently applied to validation or test observations.

This approach prevents information from the evaluation datasets from influencing the imputation statistics.

4.2 Identifier Removal

The user_id column was excluded from the predictor matrix.

Identifier fields often contain unique values and do not necessarily represent meaningful patterns that generalize to unseen observations.

Including a unique identifier can introduce noise and may encourage a model to learn patterns that do not generalize.

The target column, label, was also excluded from the predictor matrix and retained separately as the variable to be predicted.

4.3 Outlier Detection

Potential outliers were examined using the Interquartile Range (IQR) method.

The IQR is defined as:

[
IQR=Q_3-Q_1
]

where (Q_1) is the first quartile and (Q_3) is the third quartile.

The conventional lower and upper limits are:

[
L=Q_1-1.5(IQR)
]

[
U=Q_3+1.5(IQR)
]

Values outside these limits are flagged as potential outliers.

The project used this method to identify unusual observations but did not automatically delete them.

This decision is important because unusual values may contain useful information for classification. Removing them without understanding their origin could disproportionately eliminate observations belonging to the minority class.

4.4 Skewness Analysis

Feature skewness was examined to understand whether numerical distributions were approximately symmetric or strongly asymmetric.

Standard scaling was applied to models that are sensitive to feature scale, including Logistic Regression, KNN, and Linear SVM.

Tree-based models, including Decision Tree, Random Forest, and Extra Trees, were not scaled because their splitting decisions are generally not dependent on the absolute scale of numerical features.

A blanket logarithmic transformation was not applied because the anonymized features may represent different types of numerical quantities or encoded categories.

4.5 Training and Test Split

The dataset was divided into training and test partitions using an 80:20 split.

Stratified sampling was used to preserve the approximate target-class proportions in both partitions.

The training partition was used for fitting preprocessing operations, applying SMOTE, training classification algorithms, and conducting model selection.

The test partition retained the original class distribution and was not resampled.

This design is important because evaluation on an artificially balanced test set would not accurately reflect performance under the dataset's original class distribution.

4.6 Prevention of Data Leakage

The preprocessing workflow was designed to prevent data leakage by keeping imputation, scaling, and SMOTE inside the modeling pipeline.

SMOTE was applied only when fitting the training portion of the relevant pipeline.

The validation and test datasets retained their original observations and class distributions.

This ensured that synthetic minority-class observations did not directly enter the evaluation datasets.

PAGE 5: HANDLING CLASS IMBALANCE USING SMOTE

5.1 Introduction to SMOTE

The Synthetic Minority Over-sampling Technique (SMOTE) is an oversampling method that generates synthetic observations for a minority class.

Unlike random oversampling, which duplicates existing minority-class observations, SMOTE creates new observations by interpolating between minority-class examples and their nearest minority-class neighbors.

For two minority-class feature vectors (x_i) and (x_j), a synthetic observation can be generated as:

[
x_{\text{new}}=x_i+\lambda(x_j-x_i)
]

where (\lambda) is a randomly selected value between 0 and 1.

This process creates synthetic observations along the line segment between neighboring minority-class samples.

5.2 Why SMOTE Was Selected

The original dataset contains approximately 2.69% positive observations.

This imbalance can make it difficult for a classifier to learn a decision boundary that identifies minority-class observations effectively.

SMOTE was selected to increase the representation of class 1 during model training.

The intended benefits include:

- Improving minority-class representation.
- Giving classifiers more minority-class examples during training.

- Potentially improving recall and F1-score for class 1.
- Reducing dependence on simply predicting the majority class.

However, SMOTE does not guarantee better performance. Its effectiveness depends on the feature distributions, the classification algorithm, and the quality of the synthetic observations.

5.3 Application of SMOTE

SMOTE was incorporated into the imbalanced-learn modeling pipeline.

The order of operations was:

1. Impute missing numerical values.
2. Scale features where appropriate.
3. Apply SMOTE to the training observations.
4. Fit the classification algorithm.
5. Evaluate predictions on observations that were not resampled.

During cross-validation, SMOTE was applied separately within each training fold.

This prevents information leakage that could occur if oversampling were performed on the entire dataset before splitting it into training and evaluation partitions.

5.4 SMOTE and the Test Set

The test set was kept at its original class distribution.

This is essential because the objective is to evaluate the model under the distribution represented by the available data, rather than on an artificially balanced dataset.

Precision, recall, F1-score, and PR-AUC were calculated using the original test observations.

5.5 Important Assumptions and Limitations

Ordinary SMOTE assumes that interpolation between nearby minority-class observations produces meaningful synthetic data.

Because the predictor columns are anonymized, their semantic definitions are not available in this project.

If some columns represent categorical variables encoded as numbers, ordinary SMOTE may generate fractional values that do not correspond to valid categories. In such cases, SMOTENC or a different preprocessing strategy may be more appropriate.

SMOTE may also generate synthetic observations in regions where the classes overlap, potentially increasing false positives.

Therefore, the use of SMOTE should be justified by measured validation performance rather than by class imbalance alone.

PAGE 6: MODEL BUILDING AND CLASSIFICATION ALGORITHMS

6.1 Overview

Multiple supervised classification algorithms were trained and compared.

The purpose of evaluating different algorithms was to identify which approach provided the strongest balance between positive-class precision and recall while maintaining reasonable generalization.

The principal evaluation metric for model selection was the F1-score for class 1. PR-AUC, ROC-AUC, and the confusion matrix were also considered.

6.2 Logistic Regression

Logistic Regression is a linear classification algorithm that estimates the probability of an observation belonging to a particular class.

It models the relationship between predictor variables and the log-odds of the positive class.

Advantages include computational efficiency, interpretability, and a relatively simple model structure.

However, Logistic Regression may struggle when the relationship between the predictors and target is highly nonlinear.

6.3 Decision Tree Classifier

A Decision Tree partitions the feature space into regions using a sequence of decision rules.

It can model nonlinear relationships and interactions between features without requiring feature scaling.

Decision Trees are relatively easy to interpret, but individual trees can be sensitive to changes in the training data and may overfit if their complexity is not controlled.

6.4 Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines multiple decision trees.

Each tree is trained using randomized sampling and feature-selection procedures. The predictions are combined to produce a final classification.

Random Forest can capture nonlinear relationships and complex feature interactions while generally being more stable than an individual decision tree.

Its advantages include:

- Ability to model nonlinear relationships.
- Robustness to many forms of feature scaling.
- Support for complex interactions.
- Suitability for mixed patterns across numerical predictors.

Its limitations include computational cost, reduced interpretability relative to a single tree, and possible overfitting when the model is too complex.

Random Forest emerged as the strongest initial candidate in the model comparison.

6.5 K-Nearest Neighbors

KNN classifies observations based on the labels of nearby training observations.

The method is sensitive to feature scaling and the choice of the number of neighbors.

Because it calculates distances between observations, prediction can be expensive on large datasets.

The model achieved high training accuracy in the initial comparison but showed a substantial training-test accuracy gap, suggesting a generalization concern.

6.6 Linear Support Vector Machine

Linear SVM attempts to identify a decision boundary that separates classes with a large margin.

It is effective for some high-dimensional classification problems and can handle many predictors efficiently.

However, a linear decision boundary may be insufficient when the underlying class relationships are highly nonlinear.

6.7 Gaussian Naive Bayes

Gaussian Naive Bayes is a probabilistic classifier based on Bayes' theorem.

It assumes that features are conditionally independent given the class and that numerical features follow class-specific Gaussian distributions.

Although it is computationally efficient, these assumptions may not fit the relationships present in the dataset.

Its low positive-class precision indicates that it generated many false-positive predictions in this experiment.

6.8 Extra Trees Classifier

Extra Trees is an ensemble method that combines multiple randomized decision trees.

It introduces additional randomness into tree construction and can capture complex nonlinear patterns.

The model achieved comparatively high recall in the initial comparison, but its precision was substantially lower than that of Random Forest.

This suggests that it identified many positive cases but also classified a larger number of negative cases as positive.

6.9 Model Comparison Strategy

The classifiers were evaluated using a common set of metrics:

- Training accuracy.
- Test accuracy.
- Precision for class 1.
- Recall for class 1.
- F1-score for class 1.
- ROC-AUC.
- PR-AUC.
- Training-test performance gaps.

The results were used to identify the strongest candidate for further hyperparameter tuning.

PAGE 7: MODEL EVALUATION AND PERFORMANCE METRICS

7.1 Accuracy

Accuracy is the proportion of all observations that are classified correctly.

[
Accuracy=\frac{TP+TN}{TP+TN+FP+FN}
]

Here:

- (TP) represents true positives.
- (TN) represents true negatives.
- (FP) represents false positives.

- (FN) represents false negatives.

Accuracy is useful as a general measure but can be misleading for highly imbalanced datasets.

7.2 Precision

Precision measures the proportion of positive predictions that are correct.

[
Precision=\frac{TP}{TP+FP}
]

A high precision indicates that fewer negative observations are incorrectly flagged as positive.

In the final model, class 1 precision was 78.58%.

This means that approximately 79% of the observations predicted as positive were actually positive in the evaluated test set.

7.3 Recall

Recall measures the proportion of actual positive observations correctly identified by the model.

[
Recall=\frac{TP}{TP+FN}
]

The final model achieved a class 1 recall of 61.49%.

This means that approximately 61% of actual positive observations were identified. The remaining positive observations were classified as negative.

7.4 F1-score

F1-score is the harmonic mean of precision and recall.

[
F1=2\frac{Precision\times Recall}
{Precision+Recall}
]

The F1-score is useful when both false positives and false negatives matter.

The final model achieved a class 1 F1-score of 68.99%.

7.5 ROC-AUC

ROC-AUC measures the model's ability to rank positive observations above negative observations across different classification thresholds.

A value closer to 1 generally indicates stronger ranking discrimination.

The final model achieved a ROC-AUC of 0.9564.

This indicates strong discrimination in terms of ranking, although it does not guarantee that predicted probabilities are well calibrated.

7.6 Precision-Recall AUC

PR-AUC summarizes the precision-recall trade-off across different thresholds.

It is particularly useful when the positive class is rare because it focuses directly on positive-class detection.

The final model achieved a PR-AUC of 0.7203.

This metric complements ROC-AUC and provides additional information about the model's ability to identify minority-class observations.

7.7 Confusion Matrix

A confusion matrix separates predictions into four categories:

Actual / Predicted	Predicted 0	Predicted 1
Actual 0	True Negative	False Positive
Actual 1	False Negative	True Positive

The confusion matrix helps determine whether the model's errors are primarily false positives or false negatives.

In a business application, the relative cost of these errors should influence model selection and threshold choice.

7.8 Overfitting Assessment

Overfitting occurs when a model performs substantially better on training observations than on unseen data.

The project compared training and test accuracy, precision, recall, and F1-score.

The final training accuracy was 98.84%, while test accuracy was 98.52%, representing a gap of 0.32 percentage points.

However, the training F1-score was 75.74%, compared with 68.99% on the test set. The F1-score gap of 6.75 percentage points suggests that generalization is weaker for the minority class than the accuracy figures alone would indicate.

Therefore, the final model exhibits some evidence of overfitting, although the magnitude and acceptability of the gap depend on the application.

PAGE 8: HYPERPARAMETER TUNING AND THRESHOLD OPTIMIZATION

8.1 Purpose of Hyperparameter Tuning

Hyperparameters control aspects of model complexity and training behavior.

For Random Forest, relevant parameters include:

- n_estimators: number of trees.
- max_depth: maximum depth of each tree.
- min_samples_split: minimum number of observations required to split a node.
- min_samples_leaf: minimum number of observations permitted in a leaf.
- max_features: number or proportion of features considered at each split.
- class_weight: optional weighting of class-specific errors.

Hyperparameter tuning aims to identify parameter settings that improve generalization rather than simply maximize training performance.

8.2 RandomizedSearchCV

RandomizedSearchCV was used to explore combinations of Random Forest hyperparameters.

Instead of evaluating every possible parameter combination, randomized search samples a predefined number of combinations from the search space.

This reduces the computational burden compared with exhaustive grid search when the search space is large.

Stratified cross-validation was used to preserve the class distribution within the folds.

The F1-score was selected as the principal search metric because the dataset is highly imbalanced and positive-class detection is important.

The search was performed on a subset of the training data to manage computational cost.

8.3 Why the Search Used a Subset

SMOTE increases the number of training observations, which increases the computational cost of fitting Random Forest models.

Using a stratified subset allowed several parameter configurations to be explored without repeatedly processing the entire dataset.

The selected configuration was then assessed on the original test data through the subsequent evaluation workflow.

A limitation of this approach is that hyperparameters selected on a subset may not be optimal when fitted on the full training dataset.

8.4 Classification Threshold Optimization

Many classifiers generate predicted probabilities, which are converted into class labels using a threshold.

The default threshold is commonly 0.50. However, this is not necessarily the threshold that maximizes F1-score or meets a particular business requirement.

The project examined thresholds between 0.30 and 0.80 using a separate validation set.

For each threshold, precision, recall, and F1-score were calculated.

The validation results were:

Threshold	Precision	Recall	F1-score
0.30	0.5007	0.7636	0.6048
0.40	0.6159	0.7184	0.6632
0.50	0.7091	0.6702	0.6891
0.55	0.7476	0.6403	0.6898
0.60	0.7887	0.6175	0.6927
0.65	0.8230	0.5804	0.6808
0.70	0.8524	0.5393	0.6607
0.75	0.8831	0.4982	0.6370
0.80	0.9063	0.4561	0.6068

The best validation F1-score was 0.6927 at a threshold of 0.60.

Increasing the threshold generally improved precision while reducing recall. At lower thresholds, the model identified more positive observations but also generated more false positives.

The selected threshold of 0.60 provided the strongest F1-score among the tested validation thresholds.

8.5 Validation Versus Test Evaluation

Threshold optimization was performed using validation observations, while the final reported test metrics were calculated on the original test partition.

This distinction is important because repeatedly choosing thresholds using test results would make the test set part of the model-selection process.

The final test F1-score was 0.6899, compared with the best validation F1-score of 0.6927.

The small difference is encouraging, although the test set had also been consulted during earlier model comparisons. Therefore, the reported test result should be interpreted as a strong project benchmark rather than a completely untouched final estimate.

PAGE 9: FINAL MODEL RESULTS AND COMPARATIVE ANALYSIS

9.1 Initial Model Comparison

The initial model comparison produced the following results.

Model	Test Accuracy	Precision (Class 1)	Recall (Class 1)	F1-score (Class 1)	ROC-AUC	PR-AUC
Random Forest	98.13%	63.86%	70.13%	66.85%	0.9572	0.7108
Decision Tree	96.79%	43.81%	69.85%	53.85%	0.9152	0.5867
KNN	93.83%	26.54%	73.42%	38.99%	0.8728	0.3338
Extra Trees	93.22%	25.49%	79.22%	38.57%	0.9426	0.6194
Logistic Regression	86.54%	14.78%	84.21%	25.14%	0.9254	0.4642
Linear SVM	83.99%	12.72%	84.70%	22.13%	0.9152	0.4139
Naive Bayes	48.73%	4.72%	94.36%	9.00%	0.8803	0.3016

The initial Random Forest model achieved the highest F1-score among the models in this comparison.

Decision Tree was the second-strongest model based on F1-score. Several other algorithms achieved higher recall but lower precision, indicating that they classified many negative observations as positive.

The KNN training accuracy was 99.98%, compared with test accuracy of 93.83%, indicating a substantial accuracy gap in the initial comparison.

The initial model results established Random Forest as the strongest candidate for further optimization.

9.2 Final Tuned Random Forest Results

After hyperparameter tuning and threshold selection, the final reported test metrics were:

Metric	Final Result
Test Accuracy	98.52%
Precision — Class 1	78.58%
Recall — Class 1	61.49%
F1-score — Class 1	68.99%
ROC-AUC	0.9564
PR-AUC	0.7203
Classification Threshold	0.60

The tuned model's F1-score was higher than that of the initial Random Forest result. Its precision also improved, although recall decreased.

The PR-AUC increased from 0.7108 in the initial comparison to 0.7203 in the final reported evaluation.

These results suggest that the tuned model offers a better precision-recall balance for the selected threshold. However, because the workflows and thresholding procedures differ, the results should not be interpreted as a perfectly controlled head-to-head experiment.

9.3 Training and Test Performance

The tuned model's training and test performance were compared.

Metric	Training	Test	Absolute Gap
Accuracy	98.84%	98.52%	0.32 percentage points
Precision (Class 1)	86.25%	78.58%	7.67 percentage points
Recall (Class 1)	67.51%	61.49%	6.03 percentage points
F1-score (Class 1)	75.74%	68.99%	6.75 percentage points

The model maintains high accuracy on both training and test data.

However, the larger precision, recall, and F1-score gaps indicate that its ability to identify positive cases is weaker on unseen observations.

The model therefore demonstrates some overfitting, particularly with respect to minority-class classification.

9.4 Final Model Selection

The tuned Random Forest classifier with SMOTE and a threshold of 0.60 was selected as the leading candidate.

The selection was based on:

- Highest F1-score in the reported tuned evaluation.
- Strong precision for the positive class.
- Useful minority-class recall.
- Strong ROC-AUC and PR-AUC.
- High overall test accuracy.

The selection is based on the current experimental results and should be revisited if business requirements assign substantially different costs to false positives and false negatives.

PAGE 10: LIMITATIONS, FUTURE IMPROVEMENTS AND CONCLUSION

10.1 Project Limitations

Anonymized Feature Names

The predictors use anonymized column names. Without feature definitions, it is difficult to determine whether every feature represents a continuous numerical quantity, a categorical code, or another data type.

This limits domain-specific feature engineering and interpretation.

Class Imbalance

Although SMOTE increases minority-class representation during training, the final model does not identify every positive observation.

The final recall of 61.49% indicates that approximately 38.51% of actual positive cases were missed at the selected threshold.

Generalization Gap

The training F1-score exceeds the test F1-score by 6.75 percentage points.

This suggests that the model fits the training data more effectively than it generalizes to unseen observations.

Computational Constraints

Hyperparameter tuning was conducted on a subset of the training data to reduce computational cost.

The selected parameters may not be globally optimal for the complete training dataset.

Synthetic Data Assumptions

Ordinary SMOTE can create unrealistic synthetic observations when numerical features encode categorical variables or when minority-class neighborhoods contain overlapping classes.

Feature definitions should be confirmed before the model is used in a real-world setting.

Test-Set Reuse During Development

The test set was consulted during earlier model comparisons and threshold-related analysis. Although the final reported metrics are useful, this repeated use means the test set should not be regarded as a completely untouched final benchmark.

A new holdout dataset would provide a more independent estimate of final generalization performance.

10.2 Recommended Future Improvements

The following improvements could strengthen the model further.

1. Validate feature semantics.

Determine what each predictor represents and identify categorical, continuous, ordinal, or identifier variables. This will help determine whether ordinary SMOTE is appropriate or whether SMOTENC is needed.

2. Reduce overfitting.

Experiment with smaller tree depths, larger minimum leaf sizes, and other regularization parameters. Select configurations based on validation F1-score and the training-validation gap.

3. Investigate threshold alternatives.

The threshold of 0.60 maximized F1-score among the tested validation thresholds. If the application prioritizes detecting as many positive cases as possible, a lower threshold may be appropriate. If false positives are expensive, a higher threshold may be preferable.

4. Calibrate predicted probabilities.

SMOTE changes the class distribution used during training and can affect probability calibration. Calibration should be evaluated on validation data that retains the original class distribution.

5. Evaluate alternative imbalance strategies.

Compare SMOTE against class weighting, random oversampling, and other appropriate techniques using identical validation splits. This would establish whether SMOTE offers a measurable advantage.

6. Perform final independent validation.

Use a new holdout dataset or an appropriate nested cross-validation design for a less biased estimate of generalization.

7. Improve interpretability.

Once feature definitions are available, examine feature importance and use suitable interpretability methods to understand which predictors contribute most to the model's decisions.

8. Establish business-specific evaluation criteria.

Determine the costs of false positives and false negatives. This will clarify whether F1-score is the correct selection metric or whether recall, precision, or a cost-sensitive metric should take priority.

10.3 Conclusion

This project developed and evaluated a binary classification workflow for the dataset fiverr_data.csv.

The workflow included exploratory data analysis, missing-value handling, outlier inspection, feature preprocessing, stratified train-test splitting, SMOTE-based class imbalance treatment, comparison of multiple classification algorithms, cross-validation, hyperparameter tuning, and classification threshold optimization.

The initial model comparison identified Random Forest as the strongest candidate based on minority-class F1-score. Hyperparameter tuning and threshold optimization subsequently produced a tuned Random Forest model using SMOTE and a classification threshold of 0.60.

The final reported test performance was:

- Accuracy: 98.52%.
- Precision for class 1: 78.58%.
- Recall for class 1: 61.49%.

- F1-score for class 1: 68.99%.
- ROC-AUC: 0.9564.
- PR-AUC: 0.7203.

These results demonstrate that the model provides strong overall discrimination and a useful balance between precision and recall for the minority class.

However, the difference between training and test F1-score indicates some overfitting, and the current model still misses a meaningful proportion of actual positive cases.

Consequently, the tuned Random Forest with SMOTE should be considered the **best-performing candidate in the current experiment**, rather than a fully validated production model. Confirming feature semantics, evaluating the business costs of classification errors, and conducting an independent final validation would strengthen confidence in its suitability for deployment.

Overall, the project demonstrates an end-to-end machine learning classification process and highlights the importance of appropriate imbalance handling, metric selection, validation discipline, and threshold optimization when developing models for highly imbalanced datasets.